

Volume 04

Sentinel Platform Kernel (SPK)

Version: 1.0

Status: Draft

Language: English

Chapter 1

Purpose

The Sentinel Platform Kernel (SPK) is the runtime foundation of Project Sentinel. Every platform component executes through the kernel.

The kernel is responsible for:

- module lifecycle
- object lifecycle
- event routing
- authentication
- authorization
- workflows
- scheduling
- notifications
- plugin management
- health monitoring
- system diagnostics

The kernel itself shall **never perform security analysis**.

Instead, it coordinates the platform.

Chapter 2

Kernel Principles

The kernel shall be:

- Stable
- Lightweight
- Modular
- Event-driven
- Thread-safe
- Extensible
- Observable
- Secure by default

The kernel should change very rarely.

Most innovation happens in plugins and engines.

Chapter 3

Kernel Architecture

Project Sentinel

The Kernel is the central coordinator—not the intelligence layer.

Chapter 4

Kernel Responsibilities

The kernel shall never:

- scan networks
- detect vulnerabilities
- calculate risk
- analyze malware
- perform AI reasoning

Instead it shall:

- start modules
 - stop modules
 - load configuration
 - authenticate users
 - dispatch events
 - monitor health
 - schedule tasks
 - manage plugins
-

Chapter 5

Startup Sequence

Power On

If a module fails to start, the platform continues where possible and reports the failure clearly.

Chapter 6

Shutdown Sequence

Stop New Tasks

Graceful shutdown minimizes data loss and corruption.

Chapter 7

Module Registration

Every module must register itself.

Example:

Module Name

The kernel validates compatibility before activation.

Chapter 8

Health Monitoring

Every module continuously reports health.

Example metrics:

- CPU usage
- Memory usage
- Queue length
- Active workers
- Error rate
- Average execution time
- Plugin status
- API latency

The kernel aggregates these into a platform health overview.

Chapter 9

Event Routing

All communication passes through the Event Bus.

Example:

Discovery

Modules should not communicate directly unless explicitly required.

Chapter 10

Scheduler

The scheduler manages all recurring work.

Examples:

- nightly scans
- certificate checks
- cloud synchronization
- mobile synchronization
- compliance reviews
- backup verification

The scheduler supports priorities and retry policies.

Chapter 11

Configuration Management

Configuration is stored as version-controlled structured files.

Initial format:

YAML

Example:

platform:

Configuration changes are validated before activation and recorded in the audit history.

Chapter 12

Secrets Management

The kernel never stores plaintext secrets.

Sensitive values are protected using a dedicated secrets provider.

Examples:

- API keys
- SSH keys
- Cloud credentials
- Database credentials
- OAuth secrets
- Encryption keys

Secrets are referenced by identifier rather than embedded in configuration.

Chapter 13

Platform Identity

Every installation receives its own Platform ID.

Example

PS-PLT-000001

Every organization receives

PS-ORG-000001

Every user receives

PS-USR-000001

Every mission receives

PS-MSN-000001

This creates globally consistent identifiers throughout the platform.

Chapter 14

Runtime Diagnostics

The kernel continuously verifies:

- plugin compatibility
- missing dependencies
- worker failures
- memory leaks
- deadlocks (where detectable)
- queue congestion
- event processing delays
- storage availability
- API responsiveness

The goal is to identify operational issues before they affect users.

Chapter 15

Safe Mode

A unique feature.

If the kernel detects a severe failure:

- third-party plugins are disabled
- core services remain available
- diagnostics are collected
- administrators are notified
- recovery guidance is provided

This allows the platform to remain manageable even during partial failures.

Chapter 16

Platform Telemetry Policy

This is a major architectural decision.

By default:

Project Sentinel shall not send telemetry to external services without explicit administrator consent.

Possible modes:

- **Offline Mode** – No outbound telemetry.
- **Enterprise Mode** – Internal telemetry only.
- **Community Mode** – Optional anonymous usage statistics, disabled by default.

This supports privacy-conscious users, enterprises, and government deployments.

PS-ADR-0002

Title

Kernel-Centered Architecture

Status

Accepted

Decision

Project Sentinel shall implement a lightweight platform kernel responsible for orchestration, lifecycle management, security services, scheduling, and communication between modules.

The kernel shall not contain business-specific security logic.

Rationale

Separating orchestration from analysis improves maintainability, testing, extensibility, and long-term stability. Security intelligence evolves rapidly, while the kernel should remain stable for many years.

New Proposal: Engineering Validation Framework (EVF)

I would like to introduce a discipline that will guide every future design decision.

Every proposed feature must pass five validation gates before it is accepted into the Blueprint:

1. **Technical Feasibility** - Can we build it with current technology?
2. **Security Review** - Does it strengthen the platform without introducing unnecessary risk?
3. **User Value** - Does it solve a real problem for users?
4. **Maintainability** - Can a small engineering team realistically maintain it over time?
5. **Scalability** - Will it still work for a home user and for a large enterprise?

Only features that pass these gates become part of the Core Platform.

I also suggest one additional architectural rule that will influence everything we build from this point onward:

If a capability can be implemented as a plugin instead of modifying the kernel, it should be a plugin.

That single rule will help keep Project Sentinel clean, modular, and sustainable as it grows over the coming years.

====[USER]====

Remeber our system should also meet requirements for wcag. Good and continue

====[ASSISTANT]====

Excellent point.

This is exactly the kind of requirement we should define **now**, not after we start coding. This is also an example of something I want us to keep doing throughout the Blueprint:

Every important decision becomes an engineering requirement.

WCAG should not be "added later."

It becomes part of the platform architecture from Day 1.

PROJECT SENTINEL BLUEPRINT

PS-ADR-0003

Title

Accessibility by Design

Status

Accepted

Decision

Project Sentinel shall be designed to comply with modern accessibility standards from the beginning of the project.

Accessibility shall be considered an engineering requirement rather than a post-development enhancement.

Initial Target

The first production release shall target compliance with:

- WCAG 2.2 Level AA

This is a practical and widely adopted target for enterprise and public-sector applications. If future regulations or customer requirements change, we can evaluate higher conformance levels where appropriate.

Why

Accessibility improves:

- usability
- maintainability
- international adoption
- enterprise adoption
- government adoption

It also improves the overall user experience.

New Volume

Volume 05